Week of August 24, 2026:
Nesso-1 and Boltz-2 on Runs N' Poses

Yusheng Cai

August 24–30, 2026

Repository: `BindingAffinityPredictor/weeks/2026-W35`

Executive summary

This week I focused on four questions:

1. I measured the speed of Nesso-1 and Boltz-2 on the same local hardware and input cohort.
2. I used Runs N' Poses to compare Nesso-1 and Boltz-2 on the same 50 protein–ligand systems. I selected the cohort using Boltz-2's June 1, 2023 structural-training cutoff.
3. I reviewed the principal benchmarks used by the FEP community, including what data they contain, where the measurements came from, and what each benchmark tests.
4. I reviewed the Nesso-1 architecture, from protein and ligand representations through pair features, distance prediction, pocket selection, and affinity prediction.

1 Measured speed comparison

I timed both models on the new 50-system cohort using the same 10-GiB NVIDIA RTX 3080, warm local checkpoint caches, one worker, and seed 42. Nesso-1 used five recycling steps. Boltz-2 used three recycling steps, one 200-step diffusion sample, and an MSA subsample of 1,024 sequences.

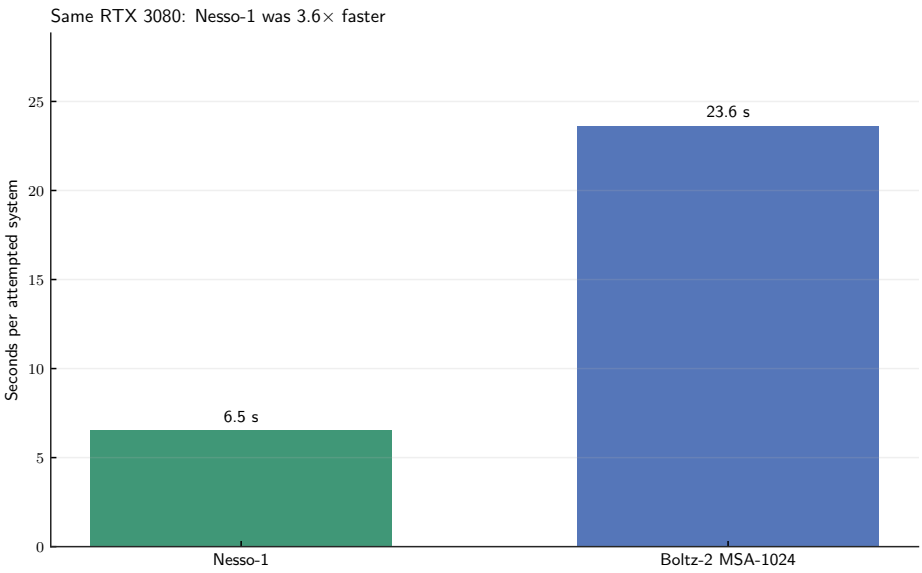

Figure 1: Measured local preprocessing and inference time divided by the 50 attempted systems. The external ColabFold MSA search is excluded.

Workflow	Completed	Total wall time	Seconds/attempt
Nesso-1	50/50	326.1 s	6.52
Boltz-2 MSA-1024	49/50	1181.7 s	23.63

Table 1: Nesso-1 was 3.62 times faster in this specific local workflow.

This is not an equal-output kernel benchmark. Boltz-2 generated full Cartesian protein–ligand complexes, whereas Nesso-1 stopped at a residue-center and ligand-heavy-atom distance representation. Boltz-2's external MSA search time is also absent. One 818-residue Boltz-2 system exceeded the available GPU memory; the failure remains in the completion count.

2 Paired Runs N' Poses experiment and results

Runs N' Poses contains 2,600 high-resolution protein–ligand systems released after September 30, 2021 [1]. It provides the experimental complex structures and precomputed similarities to older protein–ligand structures. I selected 50 systems released after June 1, 2023, with ten systems in each familiarity interval 0–20, 20–40, 40–60, 60–80, and 80–100.

2.1 Model inputs

For every system, both models received

$$\text{exact deposited protein sequence} + \text{stereochemical ligand SMILES}.$$

Boltz-2 additionally received a ColabFold MSA derived from the protein sequence.

Nesso-1 completed 50/50 systems. Boltz-2 completed 49/50; the 818-residue 8UK6 system exceeded the 10-GiB GPU limit under the frozen MSA-1024 protocol. Direct model-to-model comparisons therefore use the **same 49 complexes** completed by both models. The remaining Nesso-1-only result is retained in its completion record but excluded from paired accuracy comparisons.

2.2 What familiarity means

Runs N' Poses released a table of precomputed similarities between each test complex and candidate protein–ligand structures from the PDB [1]. For test complex i and model cutoff date c, I first exclude every candidate j released on or after the cutoff. Familiarity is the highest score among the candidates that remain:

$$F_i(c) = \max_{j:\,\mathrm{date}(j)<c} S_{ij}.$$

This is a nearest-neighbor score, not an average over the historical PDB. The released metric is

$$S_{ij} = \frac{\text{SuCOS shape}_{ij} \times \text{pocket coverage}_{ij}}{100},$$

on a 0–100 scale. SuCOS compares the two bound ligands' three-dimensional shape and chemical features; pocket coverage measures how much of the test binding pocket is matched by the candidate structure. A high value therefore means that at least one similar pocket–ligand example existed before the chosen cutoff. It is not a model prediction or an accuracy score.

Here, "cutoff" means the reported **PDB structural-training cutoff**. For Nesso-1 I use September 30, 2021; for Boltz-2 I use June 1, 2023 [2, 3]. Thus I perform two searches for the same unchanged test complex. The Boltz-2 search includes additional PDB structures released between the two dates and may therefore find a different, more similar historical neighbor. The resulting familiarity values are kept separate. They are cutoff-based proxies for what structural examples could have been available during training; they do not prove that a particular example was present in a model checkpoint.

2.3 Shared structural metrics

The closest common representation is a distance between a protein Cβ token (Cα for glycine) and a ligand heavy atom. Nesso-1 supplies an expected distance from its 64-bin distribution; Boltz-2 supplies the distance measured in its final diffusion-sampled coordinate model.

Let $t(i)$ be the representative atom of protein residue i, let j index a ligand heavy atom, and let c_b be the center of distance bin b. For Nesso-1, distogram logits ℓ^{N}_{ijb} are converted into a probability distribution and an expected distance,

$$p^{\mathrm{N}}_{ijb} = \frac{\exp(\ell^{\mathrm{N}}_{ijb})}{\sum_{b'} \exp(\ell^{\mathrm{N}}_{ijb'})}, \qquad \widehat{d}^{\,\mathrm{N}}_{ij} = \sum_b c_b p^{\mathrm{N}}_{ijb}. \tag{1}$$

For Boltz-2, the distance used in this analysis is instead measured from the predicted coordinates $\widehat{\mathbf{r}}^{\mathrm{B}}$,

$$\widehat{d}^{\,\mathrm{B}}_{ij} = \left\| \widehat{\mathbf{r}}^{\mathrm{B}}_{t(i)} - \widehat{\mathbf{r}}^{\mathrm{B}}_{j} \right\|_2. \tag{2}$$

Metric	Plain-language meaning	Better
Interface distance MAE	Mean absolute error for token pairs whose experimental distance is at most 15 Å.	Lower
Token-pair contact F1	Did the model reconstruct the detailed interface? Measures whether the correct protein token contacts the correct ligand heavy atom within 6 Å.	Higher
Residue-pocket F1	Did the model locate the correct binding region? Measures whether the correct protein residues belong to the experimental 6 Å pocket, without requiring the correct ligand-atom contact pattern.	Higher

Table 2: The three metrics applied to both model outputs. Exact ligand-graph symmetries were handled before scoring.

The reference and predicted token-pair labels are

$$y_{ij}^{\mathrm{ref}} = \mathbb{1}[d_{ij}^{\mathrm{ref}} \leq 6\ \text{Å}], \qquad \widehat{y}_{ij}^{m} = \mathbb{1}[\widehat{d}_{ij}^{m} \leq 6\ \text{Å}], \quad m \in \{\mathrm{N}, \mathrm{B}\}. \tag{3}$$

Across all scored pairs, precision, recall, and F1 are

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}. \tag{4}$$

The residue-pocket metric first asks whether each whole residue belongs to the experimental pocket. If $H(i)$ is the set of resolved heavy atoms in residue i and L is the set of ligand heavy atoms, then

$$Y_i^{\mathrm{ref}} = \mathbb{1}\left[\min_{a \in H(i),\, j \in L} \|\mathbf{r}_a^{\mathrm{ref}} - \mathbf{r}_j^{\mathrm{ref}}\|_2 \leq 6\ \text{Å}\right], \tag{5}$$

$$\widehat{Y}_i^{\mathrm{N}} = \mathbb{1}\left[\min_{j \in L} \widehat{d}_{ij}^{\mathrm{N}} \leq 6\ \text{Å}\right], \tag{6}$$

$$\widehat{Y}_i^{\mathrm{B}} = \mathbb{1}\left[\min_{a \in H(i),\, j \in L} \|\widehat{\mathbf{r}}_a^{\mathrm{B}} - \widehat{\mathbf{r}}_j^{\mathrm{B}}\|_2 \leq 6\ \text{Å}\right]. \tag{7}$$

Nesso-1 predicts reference pocket through residue-center distogram distances, whereas Boltz-2 predicts it from the final all-heavy-atom structure produced by diffusion.

2.4 Accuracy as a function of familiarity

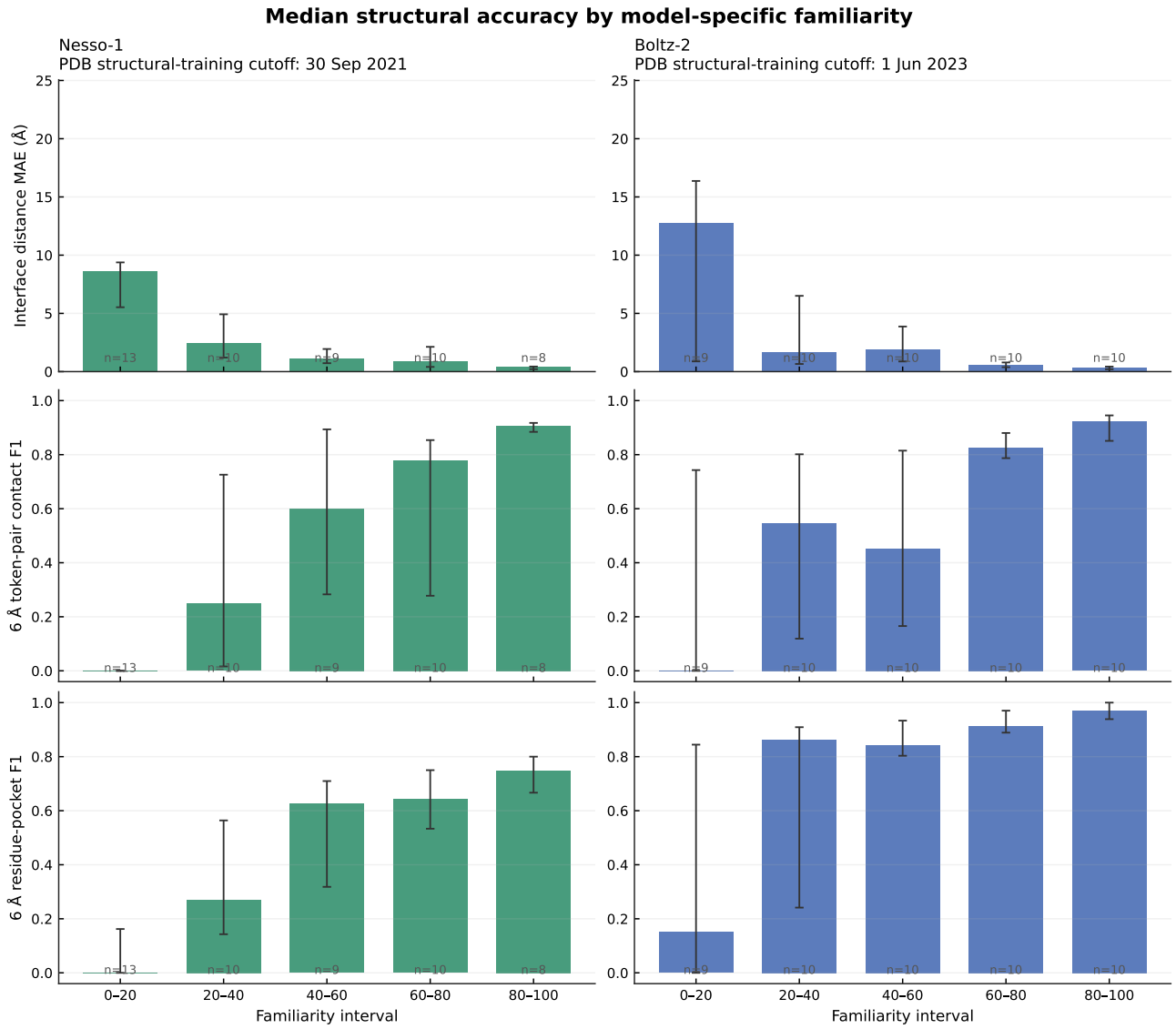

Figure 2: Structural accuracy versus model-specific familiarity. Each bar is the median in a 20-point familiarity interval; error bars are 95% bootstrap intervals, and n is the number of systems in that interval. Lower interface MAE and higher F1 are favorable.

Model and PDB structural-training cutoff	Interface MAE ρ	Token-pair F1 ρ	Residue-pocket F1 ρ
Nesso-1, Sep. 2021 ($n = 50$)	-0.721	$+0.727$	$+0.677$
	$[-0.876, -0.486]$	$[+0.511, +0.876]$	$[+0.433, +0.854]$
Boltz-2, Jun. 2023 ($n = 49$)	-0.690	$+0.692$	$+0.690$
	$[-0.839, -0.475]$	$[+0.485, +0.828]$	$[+0.472, +0.831]$

Table 3: Spearman correlations with 95% bootstrap intervals. All six intervals exclude zero.

Both models show the same high-level behavior: greater pre-cutoff structural familiarity is associated with lower interface-distance error and better contact and pocket recovery. The bin medians fluctuate, especially in low-familiarity bins; the continuous rank correlations are the primary evidence for a broad trend.

Nesso-1 improves relatively gradually across its five bins. Boltz-2's pocket recovery is already high in the 20–40 bin, but its lowest-familiarity bin contains a mixture of successes and complete failures, giving very wide uncertainty intervals. Its contact F1 and interface MAE also fluctuate between the 20–40 and 40–60 bins before improving at higher familiarity. These descriptive bin differences should not be interpreted as a head-to-head comparison at the same x-value because the two columns use different historical cutoffs and the cohort was balanced only by Boltz-2's score.

2.5 Direct comparison on the same systems

Metric	Nesso-1 median	Boltz-2 median	Boltz/Nesso/ties
Interface distance MAE (Å)	1.387	0.881	22/27/0
6 Å token-pair contact F1	0.462	0.780	27/16/6
6 Å residue-pocket F1	0.514	0.889	44/1/4

Table 4: Paired results on the 49 systems completed by both models. The final column counts which model was better on each individual system.

Boltz-2 had substantially better pocket recovery: its mean paired improvement in residue-pocket F1 was $+0.318$ with a 95% bootstrap interval of $[+0.251, +0.388]$. Token-pair contact F1 also favored Boltz-2 on average. Interface MAE was mixed: Nesso-1 was better on 27 systems, Boltz-2 on 22, and the paired mean improvement interval included zero.

Only Boltz-2 generates explicit coordinates. After global protein Cα alignment, its median protein Cα RMSD was 0.894 Å and its median symmetry-corrected ligand heavy-atom RMSD was 3.355 Å. Ligand RMSD was below 2 Å for 22/49 structures and below 5 Å for 30/49. Nesso-1 has no valid coordinate RMSD because it does not output a Cartesian complex.

3 Benchmarks used by the FEP community

Benchmark	Data vintage and provenance	Approximate scale	Main question	Paper / data
Original FEP+/JACS set	Public assay papers span roughly 2004–2013; the collection was assembled in 2015.	199 ligands, eight protein systems	Can relative binding free energies rank congeneric ligand series?	Paper; data
FEP+4 used in Week 34	Same source pool; the newest clearly linked assay series is TYK2, 2013.	87 ligands, four kinase series	Can related ligands be ranked within CDK2, JNK1, p38, and TYK2?	TYK2 source; data
D3R Grand Challenges	The latest round, GC4, used blinded Janssen and Novartis data in 2018–2019.	Challenge-dependent, blind test sets	Can methods predict poses, affinity ranks, and in some rounds relative free energies prospectively?	GC4 paper; data
Hahn/OpenFF benchmark	Many source papers; each ligand records a measurement DOI. The latest source year in the released metadata is 2019.	599 ligands, 22 systems in the published set	Can free-energy methods be compared on open, standardized inputs and measurements?	Paper; repository
Ross FEP benchmark	Prior source/FEP studies extend through 2022; 93 compounds are from anonymized in-house sets.	1,237 compounds across many series	How does FEP perform across a broader domain, including charge changes, fragments, macrocycles, scaffold hops, GPCRs, and buried-water changes?	Paper; repository
OpenFE public study	No newer compound set: it explicitly reuses the Ross v2.0 collection.	JACS, fragment, and Merck subsets of the Ross collection	Can an open RBFE workflow reproduce results on shared inputs and protocols?	Protocol; repository
CASP16 affinity task	Blinded pharmaceutical data in the 2024 challenge; public experimental-data release dated 7 August 2025.	122 assessed ligand cases across two affinity targets	In a blind challenge, can methods rank affinities before and after experimental poses are released?	Paper; data

Table 5: The benchmarks were built for different scientific purposes and should not be interpreted as interchangeable leaderboards [4–9].

4 Nesso-1 architecture, step by step

Consider one concrete input throughout this section: a protein sequence with 300 residues and *one* ligand containing 30 heavy atoms. The ligand is supplied as a SMILES string. One protein residue becomes one token and one ligand heavy atom becomes one token, so this example contains

$$N = 300 + 30 = 330 \text{ tokens.}$$

The model input does not include a protein PDB structure, an MSA, a user-specified pocket, or an initial bound pose.

The central computation for this 330-token example is:

$$\text{300-residue sequence + 30-heavy-atom ligand} \longrightarrow 330 \times 384 \text{ token features} \longrightarrow 330 \times 330 \times 128 \text{ pair features} \longrightarrow \text{distance distributions} \longrightarrow \text{predicted pocket and affinity.}$$

Nesso-1 does not first generate a complete atomic protein–ligand complex and then score that structure. Instead, it learns a probabilistic map of distances between protein-residue centers and ligand heavy atoms, then gives this learned geometry to a smaller affinity network [2].

4.1 Running example and program map

Each standard protein residue becomes one token. Its geometric target is the Cβ atom, or Cα for glycine. Each ligand heavy atom becomes one token; ligand hydrogens are omitted.

The input-processing code constructs standard reference atoms for each amino acid. For a ligand supplied as SMILES, it generates an isolated 3D conformer with RDKit ETKDG. The atom encoder uses atom identity, names, charge, chirality, hybridization, bonds, and local reference geometry, then pools atom information into token vectors.

The corresponding first step is, schematically:

```
tokens = tokenize(protein_sequence, ligand_smiles)
# 300 residue tokens + 30 ligand-heavy-atom tokens = 330 tokens

s_inputs = input_embedder(tokens)                    # [330, 384]
```

Here and below, the snippets follow the released program but omit the batch dimension, masks, normalization, dropout, and other implementation details.

4.2 ESM-2 supplies protein sequence context

The protein sequence is also processed by the pretrained ESM-2 protein language model. For each residue i, ESM-2 supplies a contextual vector

$$\mathbf{e}_i \in \mathbb{R}^{1280}.$$

"Contextual" means that the vector for an amino acid depends on the surrounding sequence, not only on its residue identity. Nesso-1 uses ESM-2 in place of the explicit MSA processing used by Boltz-2.

Separately, the atom/residue input embedder has already produced $\mathbf{s}_i^{\text{input}} \in \mathbb{R}^{384}$ for all 330 tokens. The ESM module applies a learned projection to each 1280-dimensional ESM row,

$$\widetilde{\mathbf{e}}_i = W_2\ \mathrm{ReLU}(W_1\ \mathrm{LayerNorm}(\mathbf{e}_i))\,, \qquad \mathbb{R}^{1280} \longrightarrow \mathbb{R}^{384}.$$

ESM-2 produces vectors only for protein residues, whereas Nesso's token list also contains ligand heavy atoms. The featurizer therefore creates a full token-indexed ESM array: it places each protein ESM vector in the row belonging to that protein token and leaves every ligand-token row equal to zero,

$$\mathbf{E}_{\text{full}}[i] = \begin{cases} \mathbf{e}_r^{\text{ESM}}, & i \text{ is the protein token for residue } r, \\ \mathbf{0}, & i \text{ is a ligand token.} \end{cases}$$

This is an alignment of *array indices*, not a geometric alignment of the ligand with the protein. Inside the ESM module, Nesso combines the projected ESM information with a learned projection of the atom/residue token features,

$$\mathbf{s}_i^{\text{context}} = W_{\text{input}}\mathbf{s}_i^{\text{input}} + \widetilde{\mathbf{e}}_i^{\text{ESM}}, \qquad \mathbf{s}_i^{\text{context}} \in \mathbb{R}^{384}.$$

The projection is learned: it selects and transforms the parts of ESM-2 that are useful to Nesso rather than selecting the first 384 ESM coordinates. Since the projection contains learned biases, a zero ligand row can become a shared baseline vector after projection. It still contains no ligand-specific ESM information; ligand identity and chemistry enter through the atom encoder.

For the running example, this part of the program is

$$\underbrace{300 \times 1280}_{\text{raw protein ESM-2}} \longrightarrow \underbrace{330 \times 1280}_{\substack{\text{full token-indexed ESM array} \\ \text{30 zero ligand rows}}} \longrightarrow \underbrace{330 \times 384}_{\text{projected ESM features}},$$

which is added to a separately projected 330×384 atom/residue table to form $\mathbf{S}^{\text{context}}$. The program then uses this context table to construct an additive update to the pair representation. It does not overwrite the original `s_inputs` table.

In simplified code, this stage is:

```
esm_protein = ESM2(protein_sequence)              # [300, 1280]
esm_full = zeros([330, 1280])
esm_full[protein_rows] = esm_protein              # ligand rows stay zero

esm_projected = esm_mlp(esm_full)                 # [330, 384]
s_context = project(s_inputs) + esm_projected     # [330, 384]
```

4.3 The pair representation z

The central object is not a coordinate array but a learned pair tensor

$$z \in \mathbb{R}^{N \times N \times 128}.$$

Each element z_{ij} is a 128-dimensional description of the relationship between tokens i and j. The matrix contains protein–protein, protein–ligand, ligand–protein, and ligand–ligand blocks. For example, z_{42,L_5} represents what the network currently knows about protein residue 42 relative to ligand atom 5.

The initial pair representation is built approximately as

$$z_{ij}^{(0)} = W_{\text{L}}\mathbf{s}_i^{\text{input}} + W_{\text{R}}\mathbf{s}_j^{\text{input}} + \mathbf{r}_{ij} + \mathbf{b}_{ij}.$$

The conversion from $N \times 384$ single-token features to an $N \times N \times 128$ pair tensor does not use one large MLP that directly predicts all N^2 pairs. Instead, Nesso applies two separate learned linear projections to every token,

$$\mathbf{l}_i = W_{\text{L}}\mathbf{s}_i^{\text{input}}, \qquad \mathbf{q}_i = W_{\text{R}}\mathbf{s}_i^{\text{input}}, \qquad W_{\text{L}}, W_{\text{R}} : \mathbb{R}^{384} \to \mathbb{R}^{128}.$$

This first produces two tables, each with shape $N \times 128$. Nesso then broadcasts the left table along the second token axis and the right table along the first token axis:

$$\underbrace{\mathbf{L}[:, \text{None}, :]}_{N\times 1\times 128} + \underbrace{\mathbf{Q}[\text{None}, :, :]}_{1\times N\times 128} \longrightarrow \underbrace{\mathbf{Z}^{\text{single}}}_{N\times N\times 128}, \qquad \mathbf{Z}_{ij}^{\text{single}} = \mathbf{l}_i + \mathbf{q}_j.$$

For the illustrative 330-token complex, the explicit shape change is therefore

$$330 \times 384 \longrightarrow \begin{cases} 330 \times 128 & \text{left projection,} \\ 330 \times 128 & \text{right projection} \end{cases} \longrightarrow 330 \times 330 \times 128.$$

The corresponding broadcast operation is:

```
left  = z_init_left(s_inputs)                     # [330, 128]
right = z_init_right(s_inputs)                    # [330, 128]

z_init = left[:, None, :] + right[None, :, :]    # [330, 330, 128]
z_init += relative_position_features
z_init += bond_and_bond_type_features

z = z_init
z += esm_left(s_context)[:, None, :]
z += esm_right(s_context)[None, :, :]             # ESM pair update
```

Thus the model creates one 128-dimensional vector for every ordered token pair (i, j). Because the left and right projection weights are different, z_{ij} need not equal z_{ji}.

Nesso next adds genuinely pair-specific information. The term $\mathbf{r}_{ij}$ encodes relationships such as sequence separation and chain or entity identity, while $\mathbf{b}_{ij}$ encodes ligand bonds and bond types. The ESM module also forms a second single-to-pair update: it combines the projected ESM vector with the token features, applies another left and right linear projection, broadcasts them in the same manner, and adds the result to z_{ij}. Therefore the initial pair tensor is an addition of learned token and known pair features; it is neither the scalar dot product $\mathbf{s}_i^{\mathsf{T}}\mathbf{s}_j$ nor a direct prediction of a distance.

At initialization, $z_{ij}^{(0)}$ mainly records what kinds of objects i and j are and their known sequence or bond relationship. Its useful geometric meaning is developed by the Pairformer. In particular, the broadcasted sum $\mathbf{l}_i + \mathbf{q}_j$ is initially a separable description of the two tokens. The following Pairformer blocks allow pair (i, j) to interact with other pairs through intermediate tokens k, producing the more complex context-dependent relationships needed to infer contacts and distances.

4.4 What the Pairformer does

The released trunk contains 48 Pairformer blocks. Every block applies, in order:

1. outgoing triangle multiplication;
2. incoming triangle multiplication;
3. starting-node triangle attention;
4. ending-node triangle attention; and
5. a pair-transition multilayer perceptron.

The following pseudocode closely follows the released `PairformerNoSeqLayer.forward` method:

```
def pairformer_block(z, pair_mask):                # z: [330, 330, 128]
    update = triangle_multiply_outgoing(z, pair_mask)
    z = z + dropout(update)

    update = triangle_multiply_incoming(z, pair_mask)
    z = z + dropout(update)

    update = triangle_attention_starting(z, pair_mask)
    z = z + dropout(update)

    update = triangle_attention_ending(z, pair_mask)
    z = z + columnwise_dropout(update)

    z = z + pair_transition_mlp(z)                 # 128 -> 512 -> 128
    return z                                       # [330, 330, 128]

for block in pairformer_blocks:                   # 48 blocks
    z = pairformer_block(z, pair_mask)
```

Each operation produces a residual update, $z \leftarrow z + \Delta z$, so the network progressively refines rather than replaces the pair table. The `pair_mask` prevents padded or discarded token pairs from contributing.

Dropout regularizes training and is disabled during ordinary inference; it is not an additional source of randomness in a standard evaluation run.

At the program level, the initialization, ESM update, and Pairformer are repeated during recycling:

```
z = zeros_like(z_init)
for pass_index in range(recycling_steps + 1):
    z = z_init + recycle_projection(normalize(z))
    z = esm_module(z, s_inputs, esm_full, pair_mask)
    z = pairformer(z, pair_mask)                    # 48 blocks
```

Thus `recycling_steps=5` means six trunk evaluations: one initial evaluation and five recycled evaluations.

Triangle multiplication. To update pair (i, j), triangle multiplication aggregates information through every possible third token k. Its two directions can be written schematically as

$$\Delta z_{ij}^{\text{out}} \sim \sum_k A(z_{ik}) \odot B(z_{jk}), \qquad \Delta z_{ij}^{\text{in}} \sim \sum_k A(z_{ki}) \odot B(z_{kj}),$$

where A and B are learned projections and $\odot$ denotes elementwise multiplication. These equations show the information flow rather than every normalization and gate in the implementation.

The released multiplication layers can be reduced to the following pseudocode. The only difference between the two directions is which two sides of the triangle are contracted over k:

```
def prepare_triangle_inputs(z, pair_mask):
    x = layer_norm(z)                                # [N, N, 128]
    x_gated = linear_256(x) * sigmoid(gate_256(x))
    x_gated *= pair_mask[..., None]
    a, b = split_in_half(x_gated)                   # each [N, N, 128]
    return x, a, b

def triangle_multiply_outgoing(z, pair_mask):
    x, a, b = prepare_triangle_inputs(z, pair_mask)
    triangle = einsum("ikd,jkd->ijd", a, b)         # sum over k
    update = output_linear(layer_norm(triangle))
    return update * sigmoid(output_gate(x))

def triangle_multiply_incoming(z, pair_mask):
    x, a, b = prepare_triangle_inputs(z, pair_mask)
    triangle = einsum("kid,kjd->ijd", a, b)         # sum over k
    update = output_linear(layer_norm(triangle))
    return update * sigmoid(output_gate(x))
```

For example, let i be protein residue 42, j ligand atom 5, and k ligand atom 6. If atoms 5 and 6 are bonded and atom 6 is predicted near residue 42, the two other sides of the triangle provide information about the possible relationship between residue 42 and atom 5.

Triangle attention. Attention also considers third tokens k, but learns which intermediates are most relevant. One attention direction is schematically

$$\alpha_{ijk} = \text{softmax}_k\left(\frac{\mathbf{q}_{ij}^{\mathsf{T}}\mathbf{k}_{ik}}{\sqrt{d}} + \text{pair bias}\right), \qquad \Delta z_{ij} = \sum_k \alpha_{ijk}\mathbf{v}_{ik}.$$

The released blocks use four attention heads, allowing different heads to emphasize different relational patterns. Dot products therefore occur *inside* attention when calculating weights, but the complete z tensor is not a dot-product similarity matrix.

The corresponding attention logic is:

```
def triangle_attention_starting(z, pair_mask):
    x = layer_norm(z)
    Q, K, V = project_to_four_attention_heads(x)  # head width = 32
    pair_bias = project_pair_bias(x)

    logits = dot(Q[i, j], K[i, k]) / sqrt(32)
    logits += pair_bias[i, j, k]
    logits += mask_bias(pair_mask[i, k])

    weights = softmax(logits, over="k")
    update[i, j] = sum_k(weights[i, j, k] * V[i, k])
    return combine_attention_heads(update)

def triangle_attention_ending(z, pair_mask):
    z_T = transpose_token_axes(z)
    mask_T = transpose_token_axes(pair_mask)
    update_T = triangle_attention_starting(z_T, mask_T)
    return transpose_token_axes(update_T)
```

Pair transition. After communication among pairs, a small MLP transforms the 128 features within each pair independently,

$$z_{ij} \leftarrow z_{ij} + \mathrm{MLP}(z_{ij}).$$

Triangle operations communicate across the pair table; the transition mixes features within one table entry.

In the released transition, two separate 512-dimensional projections are multiplied to gate the hidden representation:

```
def pair_transition_mlp(z):                          # [N, N, 128]
    x = layer_norm(z)
    hidden = silu(linear1_512(x)) * linear2_512(x)
    update = linear3_128(hidden)
    return update                                    # [N, N, 128]
```

The Pairformer does not analytically enforce molecular geometry. Instead, the distogram training loss propagates through all 48 blocks, rewarding internal features that predict experimental distance bins. This supervision is what makes the final z geometrically informative. Before the distogram head, the released implementation symmetrizes it as

$$z_{ij}^{\mathrm{sym}} = z_{ij} + z_{ji},$$

consistent with the physical symmetry $d_{ij} = d_{ji}$.

Unlike a full AlphaFold-style trunk, the Nesso-1 trunk updates only z. It does not maintain a large evolving MSA representation and it does not send z into an atomic diffusion decoder.

4.5 From z to a distogram

A linear head converts every pair representation into 64 logits. Softmax gives a categorical distance distribution,

$$p_{ij}^{(b)} = P(d_{ij} \text{ lies in distance bin } b)\,.$$

In code, the last axis contains the 64 alternatives for each pair:

```
z_symmetric = z + transpose_token_axes(z)         # [330, 330, 128]
distance_logits = distogram_head(z_symmetric)     # [330, 330, 64]
distance_probabilities = softmax(distance_logits, dim=-1)
```

Thus, the model can assign probability to several possible distances instead of committing to one coordinate set. Using bin center c_b, its expected distance is

$$\bar{d}_{ij} = \sum_b p_{ij}^{(b)} c_b.$$

The uncertainty of a pair distribution can be summarized by

$$H_{ij} = -\sum_b p_{ij}^{(b)} \log p_{ij}^{(b)}.$$

Nesso reports averaged protein–protein, protein–ligand, and ligand–ligand entropies. A concentrated distance distribution has lower entropy; a diffuse one has higher entropy.

This distogram is Nesso-1's native structural output. The paper reconstructs approximate residue-center and ligand-heavy-atom poses by optimizing coordinates to agree with the expected distances, but the standard released inference path does not generate a full atomic protein PDB structure.

4.6 Automatic pocket selection and recycling

The model infers the relevant protein region rather than receiving a pocket from the user:

1. The first trunk pass processes the complete protein and ligand.
2. The distogram head estimates each protein residue's distance to the ligand.
3. Protein tokens predicted within 22 Å of the ligand are retained, subject to a default total crop budget of 256 tokens.
4. Later recycling passes repeatedly update z using this smaller region.

Schematically, the first pass triggers the crop and all related arrays are cropped using the same retained token indices:

```
if pass_index == 0 and refine_pocket:
    keep_tokens = pocket_crop(z, distance_cutoff=22, token_budget=256)
    z_init   = z_init[keep_tokens][:, keep_tokens]
    s_inputs = s_inputs[keep_tokens]
    esm_full = esm_full[keep_tokens]
```

Our command used `recycling_steps=5`. In the released implementation, this means one initial trunk evaluation followed by five recycled evaluations. For the affinity calculation, a tighter predicted cutoff of 15 Å selects the protein tokens supplied to the affinity modules. These cutoffs define model context; they should not be interpreted as exact physical boundaries of the binding site.

4.7 The affinity modules

The selected interface is passed to a second, smaller Pairformer. It receives:

- token features from the atom and residue encoders;
- the ESM-2 protein embeddings;
- the pretrained pair representation z; and
- the predicted distance-bin probabilities.

The released checkpoint contains two independently parameterized affinity modules, each with eight Pairformer blocks. Their masks focus computation on protein–ligand and ligand–ligand pairs. After these updates, the relevant pair features are averaged into one global vector,

$$\mathbf{g} = \frac{\sum_{ij} m_{ij} z_{ij}}{\sum_{ij} m_{ij}},$$

and small multilayer perceptrons map $\mathbf{g}$ to a continuous affinity output and a binary binder output.

The two continuous module outputs are

$$\texttt{affinity\_pred\_value1} \qquad \texttt{affinity\_pred\_value2}.$$

The value normally used is their mean,

$$\texttt{affinity\_pred\_value} = \frac{\texttt{value1} + \texttt{value2}}{2}.$$

The released model logic can therefore be summarized as:

```
value1, binder1 = affinity_module1(z, distance_probabilities, inputs)
value2, binder2 = affinity_module2(z, distance_probabilities, inputs)

affinity_pred_value = (value1 + value2) / 2
binder_probability  = (binder1 + binder2) / 2
```

The two binder probabilities are likewise averaged. The released documentation expresses the continuous value as

$$\log_{10}\left(\frac{\mathrm{IC}_{50}}{\mu\mathrm{M}}\right),$$

so lower values indicate stronger predicted binding: -3, 0, and 2 correspond nominally to approximately 1 nM, 1 μM, and 100 μM. Because the training labels mix several assay endpoints, this output is best treated as an assay-trained potency score, not as a rigorous calculated IC_{50} or binding free energy.

The binary output is

$$P(\text{binder} \mid \text{protein, ligand}),$$

which addresses classification rather than continuous potency ranking. The two outputs should remain separate in evaluation.

4.8 Training objectives

Training is most easily understood in two stages. First, the atom encoder and trunk learn geometry by minimizing categorical cross-entropy for experimental distance bins. Structural training uses PDB structures released before September 30, 2021, AFDB protein distillation data, and distilled protein–ligand complexes described in the paper. This teaches z to encode pairwise geometry.

Second, the affinity modules learn public assay outcomes. The regression data come mainly from BindingDB and ChEMBL, while classification data also include PubChem, the CeMM fragment collection, and MIDAS. Classification uses focal loss. Regression uses Huber losses for both absolute values and within-assay differences. The relative term encourages

$$\hat{y}_A - \hat{y}_B \approx y_A - y_B$$

for compounds measured in the same assay. It is up-weighted because public assays can have different systematic offsets, while within-assay ligand ordering is often the more reliable medicinal-chemistry signal.

Affinity and potency labels K_d, K_i, IC_{50}, and EC_{50} are treated interchangeably during this training. This choice expands the public training set but limits the strict physical interpretation of the continuous output.

4.9 Putting everything together

Table 6 returns to the running example and follows the major arrays in program order. The first trunk pass uses all 330 tokens. When pocket refinement is enabled, it then retains $M \leq 256$ tokens and later passes operate on the smaller $M \times M$ pair table.

Step	Program object	Shape in the running example	Meaning
1	Token records	$300 + 30 = 330$ tokens	One token per protein residue and ligand heavy atom
2	`s_inputs`	330×384	Atom/residue identity and local chemical features
3	`esm_full` and ESM projection	$300 \times 1280 \to 330 \times 1280 \to 330 \times 384$	Protein sequence context; ligand ESM rows begin as zero
4	`z_init`	$330 \times 330 \times 128$	Ordered token-pair features from token projections, relative position, and bonds
5	First Pairformer pass	$330 \times 330 \times 128$	Context-dependent pair relationships after 48 blocks
6	Predicted pocket crop	$330 \to M$, with $M \leq 256$	Ligand plus protein context predicted within the 22 Å crop rule
7	Recycled pair table and distogram	$M \times M \times 128$ and $M \times M \times 64$	Refined pair features and 64 distance-bin probabilities per pair
8	Affinity modules	two continuous values and two binder probabilities	Two eight-block affinity networks; each pair of outputs is averaged

Table 6: Shape ledger for the 300-residue protein and 30-heavy-atom ligand. M is the number of tokens retained by automatic pocket cropping.

The complete pocket-refined calculation can now be read as the following high-level program. Each helper function corresponds to code and equations shown in the preceding subsections.

```
def nesso_predict(protein_sequence, ligand_smiles, recycling_steps=5):
    # 1. Token and single-token features
    tokens = tokenize(protein_sequence, ligand_smiles)       # 330 tokens
    s_inputs = input_embedder(tokens)                        # [330, 384]

    # 2. Protein language-model context
    esm_protein = ESM2(protein_sequence)                     # [300, 1280]
    esm_full = place_in_protein_rows(esm_protein, N=330)     # [330, 1280]
    s_context = project(s_inputs) + esm_mlp(esm_full)        # [330, 384]

    # 3. Initial pair table: token projections + r_ij + b_ij
    z_init = initialize_pairs(s_inputs, relative_features, bond_features)
                                                              # [330, 330, 128]
    z = zeros_like(z_init)

    # 4. One initial pass plus five recycled passes
    for pass_index in range(recycling_steps + 1):
        z = z_init + recycle_projection(normalize(z))
        z += single_to_pair_update(s_context)

        for block in pairformer_blocks:                      # 48 blocks
            z = pairformer_block(z, pair_mask)

        if pass_index == 0:                                  # full 330 tokens
            p_full = softmax(distogram_head(z + z.transpose(0, 1)))
            keep = select_pocket(p_full, cutoff=22, budget=256)
            z, z_init, s_inputs, esm_full, pair_mask = crop_all(keep)
            s_context = rebuild_context(s_inputs, esm_full)

    # 5. Final structural probabilities and predicted interface
    p_distance = softmax(distogram_head(z + z.transpose(0, 1)))
    interface = select_affinity_interface(p_distance, cutoff=15)

    # 6. Two affinity estimates and two binder classifications
    affinity_inputs = crop_affinity_inputs(
        z, p_distance, s_inputs, esm_full, interface
    )
    value1, binder1 = affinity_module1(**affinity_inputs)
    value2, binder2 = affinity_module2(**affinity_inputs)

    return {
        "distance_probabilities": p_distance,
        "pocket_crop_indices": keep,              # 22-A context crop
        "affinity_interface": interface,          # tighter 15-A selection
        "affinity_pred_value": (value1 + value2) / 2,
        "binder_probability": (binder1 + binder2) / 2,
    }
```

The key scientific interpretation is that Nesso-1 finishes with a probability distribution over pairwise distances, a predicted protein–ligand interface, an assay-trained continuous potency score, and a binder

probability. The $N \times N$ or $M \times M$ tables are learned relationships, not Cartesian coordinates. Unless a separate distance-based reconstruction procedure is run, this calculation does not produce a complete all-atom bound PDB structure, and the continuous score is not a physics-derived binding free energy.

5 Conclusion and limits

The paired experiment gives two main results. First, structural familiarity is strongly associated with accuracy for both Nesso-1 and Boltz-2, using a model-appropriate historical cutoff for each. Second, Boltz-2's explicit coordinate generation recovers the experimental pocket more reliably, whereas Nesso-1 is faster and remains competitive on the common interface-distance error metric.

The cohort was deliberately balanced by Boltz-2's familiarity score and is not a random sample of all Runs N' Poses systems. The calculation uses one seed and one Boltz-2 diffusion sample per system. MSA-1024 is a local memory-constrained protocol, and one long protein failed. Finally, Runs N' Poses does not provide a uniform affinity endpoint for these systems, so none of these structural results establishes affinity-prediction accuracy.

The benchmark survey also changes how I interpret the Week 34 affinity result. The 87 compounds came from only four kinase series, so that calculation is a useful released-model reproduction and a test of within-series ranking, but not a broad test of target-level generalization. The larger public FEP collections provide more diverse follow-up systems, while their prepared structures and congeneric series must remain part of the interpretation.

For next week, I will apply the *Runs N' Poses idea* to affinity benchmarks rather than treating the FEP benchmark names as sufficient evidence of difficulty. Beginning with Nesso-1 and its September 2021 structural cutoff, I will determine how similar each FEP test pocket and ligand is to complexes available before that cutoff, first for FEP+4 and then for selected open Hahn/OpenFF or Ross series. Pocket familiarity, ligand chemical familiarity, and affinity-training overlap will be recorded separately. I will then ask whether Nesso-1's within-series ranking performance changes with familiarity. This is planned work; no such FEP familiarity result is claimed in this report.

References

[1] Peter Škrinjar, Jérôme Eberhardt, Janani Durairaj, and Torsten Schwede. Evaluating generalization in protein–ligand cofolding methods. *Nature Structural & Molecular Biology*, 33:782–794, 2026. doi: 10.1038/s41594-026-01797-5. URL `https://doi.org/10.1038/s41594-026-01797-5`.

[2] Nikhil Shenoy, David Errington, Emmanuel Bengio, Kacper Kapuśniak, Kerstin Klaeser, Yui Tik Pang, Vladimir Radenkovic, Prudencio Tossou, Therence Bois, Andrew Wedlake, and Francesco Di Giovanni. Nesso-1: Accelerating open-source binding affinity predictions. *bioRxiv*, 2026. doi: 10.64898/2026.08.01.742196. URL `https://doi.org/10.64898/2026.08.01.742196`. Preprint, version 1, posted 3 August 2026.

[3] Saro Passaro, Gabriele Corso, Jeremy Wohlwend, Mateo Reveiz, Stephan Thaler, Vignesh Ram Somnath, Noah Getz, Tally Portnoi, Julien Roy, Hannes Stark, David Kwabi-Addo, Dominique Beaini, Tommi Jaakkola, and Regina Barzilay. Boltz-2: Towards accurate and efficient binding affinity prediction. *bioRxiv*, 2025. doi: 10.1101/2025.06.14.659707. URL `https://doi.org/10.1101/2025.06.14.659707`. Preprint, version 1, posted 18 June 2025.

[4] Lingle Wang, Yujie Wu, Yuqing Deng, Byungchan Kim, Levi Pierce, Goran Krilov, Dmitry Lupyan, Shaughnessy Robinson, Markus K. Dahlgren, Jeremy Greenwood, Donna L. Romero, Craig Masse, Jennifer L. Knight, Thomas Steinbrecher, Thijs Beuming, Wolfgang Damm, Edward Harder, Woody Sherman, Mark Brewer, Ronald Wester, Mark Murcko, Leah Frye, Ramy Farid, Teng Lin, David L.

Mobley, William L. Jorgensen, Bruce J. Berne, Richard A. Friesner, and Robert Abel. Accurate and reliable prediction of relative ligand binding potency in prospective drug discovery by way of a modern free-energy calculation protocol and force field. *Journal of the American Chemical Society*, 137(7):2695–2703, 2015. doi: 10.1021/ja512751q. URL https://doi.org/10.1021/ja512751q.

[5] David F. Hahn et al. Best practices for constructing, preparing, and evaluating protein-ligand binding affinity benchmarks. *Living Journal of Computational Molecular Science*, 4(1):1497, 2022. doi: 10.33011/livecoms.4.1.1497. URL https://doi.org/10.33011/livecoms.4.1.1497.

[6] Gregory A. Ross, Chao Lu, Guido Scarabelli, Steven K. Albanese, Evelyne Houang, Robert Abel, Edward D. Harder, and Lingle Wang. The maximal and current accuracy of rigorous protein–ligand binding free energy calculations. *Communications Chemistry*, 6:222, 2023. doi: 10.1038/s42004-023-01019-9. URL https://doi.org/10.1038/s42004-023-01019-9.

[7] Open Free Energy Consortium. OpenFE industry benchmark 2024: Public dataset benchmarks. Online documentation, 2024. URL https://industrybenchmarks2024.readthedocs.io/en/latest/public/overview.html. Accessed 30 August 2026.

[8] Michael K. Gilson, Jerome Eberhardt, Peter Škrinjar, Janani Durairaj, Xavier Robin, and Andriy Kryshtafovych. Assessment of pharmaceutical protein–ligand pose and affinity predictions in CASP16. *Proteins: Structure, Function, and Bioinformatics*, 94(1):249–266, 2026. doi: 10.1002/prot.70061. URL https://doi.org/10.1002/prot.70061.

[9] Zied Gaieb, Conor D. Parks, Michael Chiu, Huanwang Yang, Chenghua Shao, W. Patrick Walters, Millard H. Lambert, Neysa Nevins, Scott D. Bembenek, Michael K. Ameriks, Tara Mirzadegan, Stephen K. Burley, Rommie E. Amaro, and Michael K. Gilson. D3R grand challenge 3: Blind prediction of protein–ligand poses and affinity rankings. *Journal of Computer-Aided Molecular Design*, 33(1):1–18, 2019. doi: 10.1007/s10822-018-0180-4. URL https://doi.org/10.1007/s10822-018-0180-4.